Project - High Level Design on

Autonomous Logistics Researcher Agent

Course Name: Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	Vasu Joshi	EN22EL301063
2.	Yash Kag	EN22CS3011103
3.	Yash Parashar	EN22CS3011106
4.	Yashee Verma	EN22CS3011115
5.	Yashraj Singh Sisodiya	EN22CS3011120

Group Name: Group 12D10

Project Number: AAI-25

Industry Mentor Name:

University Mentor Name: Prof. Maya Baniya

Academic Year: 2025-2026

Table of Content

1. Introduction.
   1.1 Scope of the document.
   1.2 Intended Audience
   1.3 System overview.

2. System Design.
   2.1 Application Design
   2.2 Process Flow.
   2.3 Information Flow.
   2.4 Components Design
   2.5 Key Design Considerations
   2.6 API Catalogue.

3. Data Design.
   3.1 Data Model
   3.2 Data Access Mechanism
   3.3 Data Retention Policies
   3.4 Data Migration

4. Interfaces

5. State and Session Management

6. Caching

7. Non-Functional Requirements
   7.1 Security Aspects
   7.2 Performance Aspects

8. References

1. Introduction:

The *Autonomous Logistics Researcher Agent* is an advanced AI-driven system designed to automate research tasks within the logistics domain. Leveraging Agentic AI principles, the system uses Large Language Models (LLMs) along with frameworks like LangChain and CrewAI to independently process user queries, retrieve relevant information from the web, and generate structured insights.
This project aims to reduce manual research effort, improve decision-making, and provide reliable, real-time logistics intelligence by integrating autonomous reasoning, tool usage, and memory capabilities.

1.1 Scope of the document:

This document provides a comprehensive overview of the system design and implementation of the Autonomous Logistics Researcher Agent. It includes:

- System architecture and workflow
- Functional and non-functional aspects
- Component-level design
- Data and information flow
- API interactions and tools used

1.2 Intended Audience:

This document is intended for:

- Project evaluators and professors
- Developers and AI engineers
- Researchers in logistics and AI
- Stakeholders interested in automation and intelligent systems

1.3 System overview:

The system is an AI-powered autonomous agent that:

- Accepts user queries related to logistics (e.g., supply chain trends, transportation optimization)
- Uses integrated tools (web search APIs) to gather information
- Processes and summarizes findings using LLMs
- Stores structured outputs into a knowledge repository (text-based)

Core Features:

- Autonomous decision-making
- Multi-source information synthesis
- Context-aware summarization
- Persistent knowledge storage

2 System Design:

The system follows a modular and agent-based architecture:
Architecture Layers:

- **User Interface Layer** – Input query interface
- **Agent Layer (CrewAI)** – Manages tasks and agent roles
- **Orchestration Layer (LangChain)** – Handles tool chaining and workflows
- **LLM Layer** – Processes natural language and generates responses
- **Tool Layer** – Web search, scraping tools
- **Storage Layer** – Saves summarized results

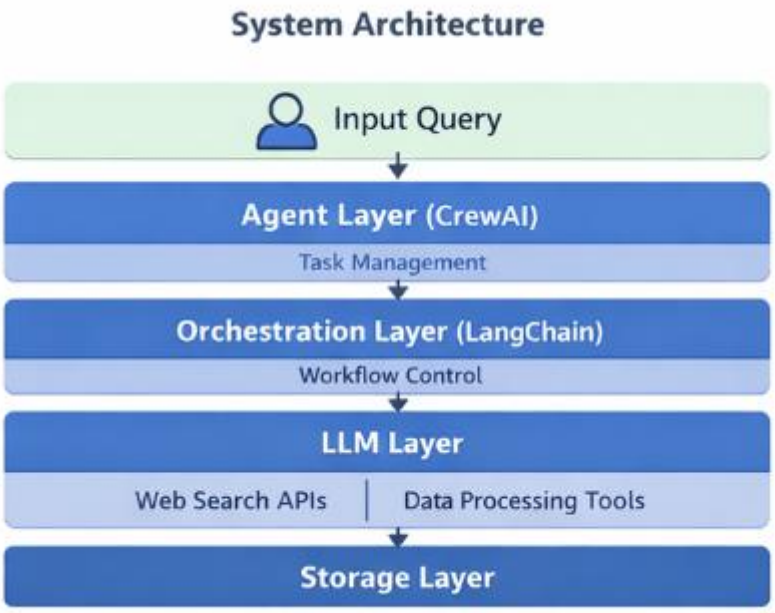

Fig 1: System Architecture

2.1 Application Design:

The application is designed using an agent-task model:

- Research Agent → Searches and collects data
- Analysis Agent → Processes and filters relevant information
- Summarization Agent → Generates concise insights
- Storage Agent → Saves results to repository

Each agent collaborates using CrewAI workflows, enabling parallel and sequential task execution.

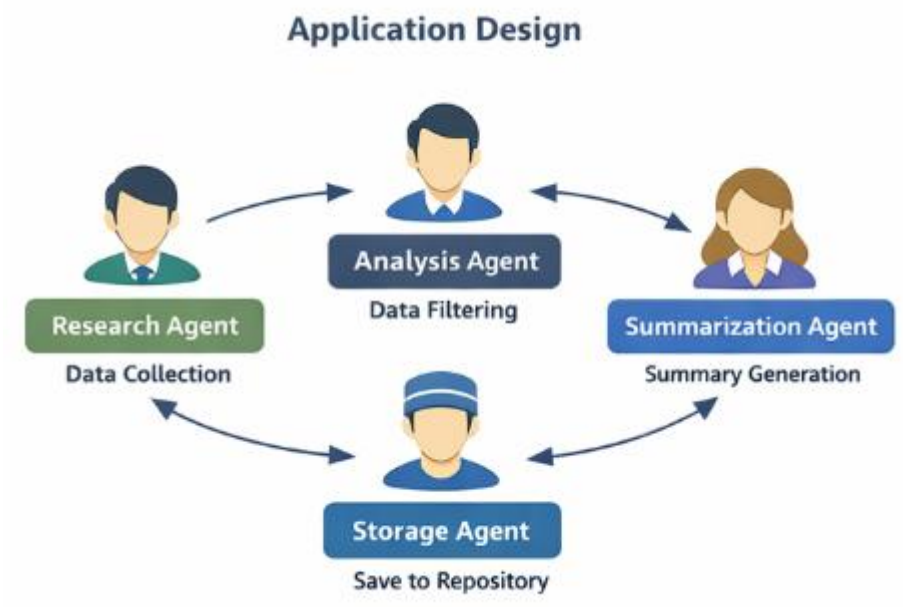

Fig 2: Application Design

2.2 Process Flow:

- User submits a logistics-related query
- Agent interprets query using LLM
- Web search tool retrieves relevant data
- Data is filtered and validated
- LLM summarizes insights
- Output is formatted
- Results stored in a text repository
- Final response displayed to user

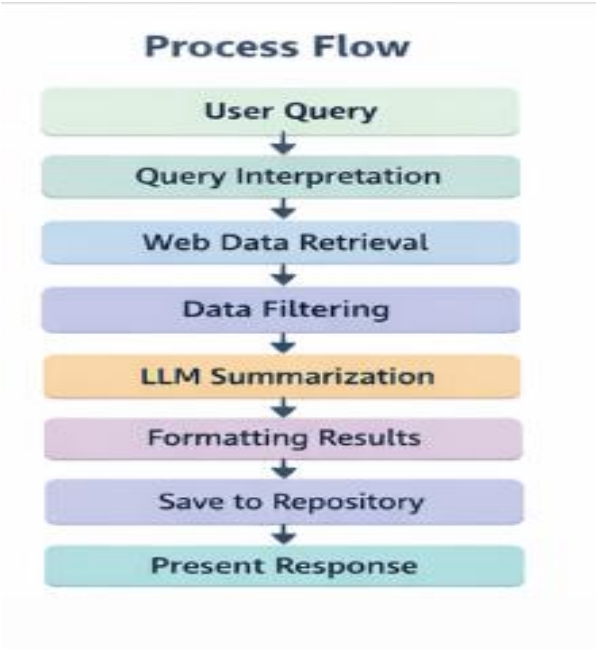

Fig 3: Process Flow

2.3 Information Flow:

- Input: User query
- Processing:
  Query → LLM → Tool selection
  Tool → Data retrieval → LLM processing
- Output:
  Structured summary
  Stored knowledge file

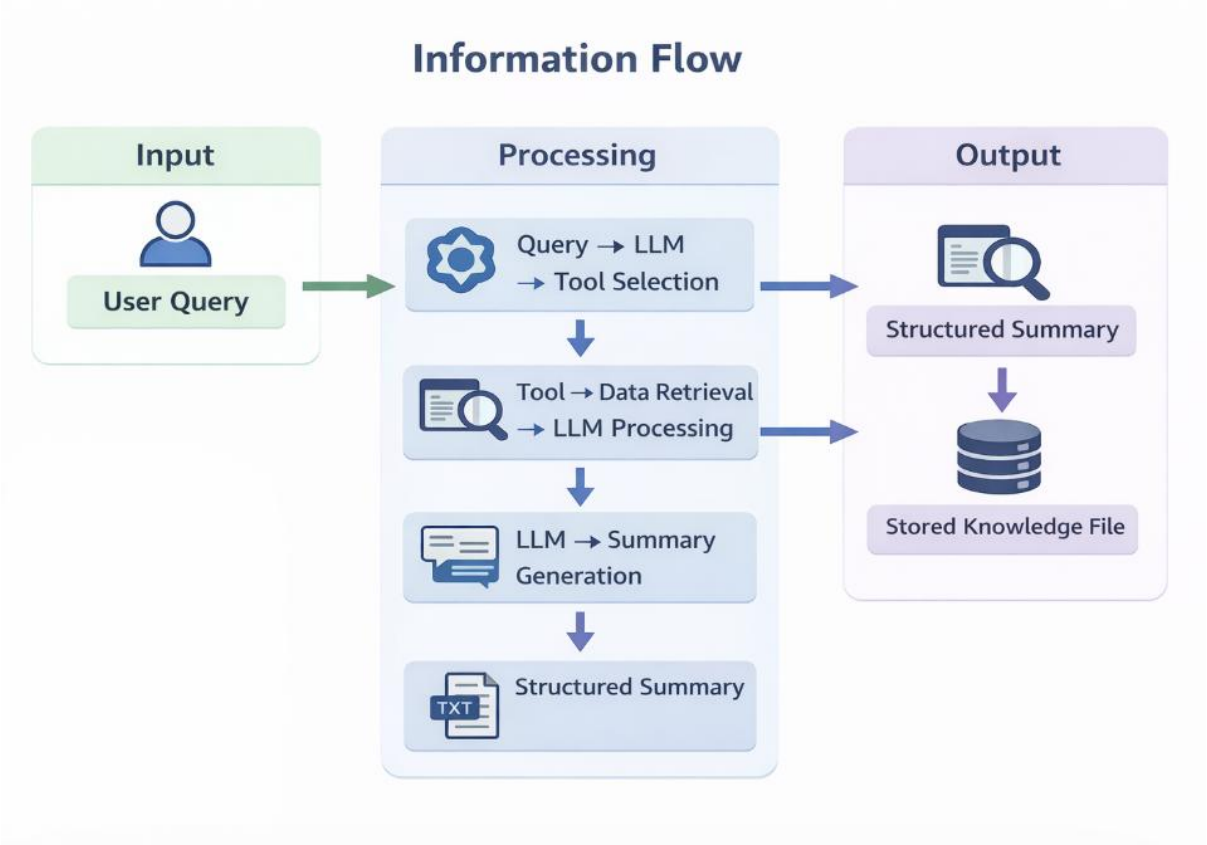

Fig 4: Information Flow

2.4 Components Design:

1. Query Processor

- Parses user input
- Identifies intent and keywords

2. Agent Manager (CrewAI)

- Assigns tasks to agents
- Coordinates execution

3. Tool Integrator

- Connects to web search APIs
- Retrieves real-time data

4. LLM Engine

- Generates summaries
- Performs reasoning

5. Storage Module

- Saves output in .txt or structured format

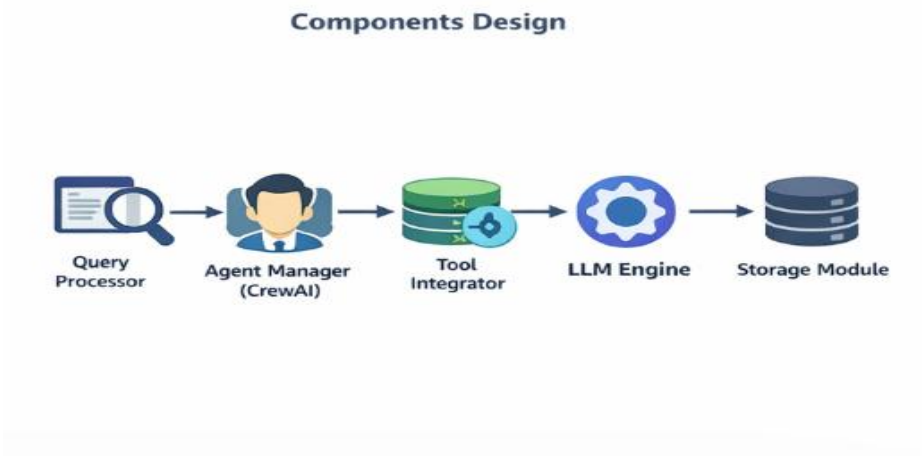

Fig 5: Components Design

2.5 Key Design Considerations:

- **Scalability:** Modular architecture for future expansion
- **Accuracy:** Use of multiple sources for reliable output
- **Autonomy:** Minimal human intervention
- **Performance:** Efficient query handling
- **Security:** Safe API usage and data handling
- **Extensibility:** Easy integration of new tools and agents

2.6 API Catalogue:

- **LLM API (OpenAI / others):** Natural language processing
- **Web Search API**: Retrieve real-time logistics data
- **LangChain:** Workflow orchestration
- **CrewAI**: Multi-agent coordination
- **File System API**: Data storage

3 Data Design:

The system handles:

- User queries (input)
- Retrieved web data (intermediate)
- Summarized outputs (final)
- Metadata (timestamps, sources)

Formats include JSON (processing) and TXT/Markdown (storage).

3.1 Data model:

Entities:

- Query
- Research Data
- Summary
- Knowledge Repository

Relationships:

- One query → multiple data sources
- Data → processed into summary
- Summary → stored in repository

3.2 Data Access Mechanism:

- API-based data retrieval
- LangChain tool integration
- File system access for storage
- Temporary memory for context handling

3.3 Data Retention Policies:

- Temporary data removed after processing
- Final summaries stored persistently
- Logs maintained for debugging
- Optional automatic cleanup policies

3.4 Data Migration:

- Supports migration to cloud databases
- Export formats: JSON, TXT
- Backup mechanisms for safe transfer

4. Interfaces :

- **User Interface:** Query input and result display
- **System Interfaces:** APIs for LLM and web search
- **Agent Interfaces:** Communication between agents

5. State and Session Management :

- Maintains session context for queries
- Supports short-term memory for reasoning
- Can operate in stateless or session-based mode

6. Caching :

- Stores frequently used queries/results
- Reduces API usage
- Improves performance
- Implemented using memory or tools like Redis

7. Non-Functional Requirements :

- Scalability for multiple users
- Reliability with fault tolerance
- Usability with simple interface
- Maintainability with modular design

7.1 Security Aspects :

- Secure API key storage
- Input validation
- Data encryption (if cloud-based)

- Access control mechanisms

7.2 Performance Aspects :

- Fast query processing
- Parallel agent execution
- Reduced latency using caching
- Efficient data filtering

8. References:

- . LangChain Documentation : *Home - Docs by LangChain*
- CrewAI Framework Docs : *CrewAI Documentation - CrewAI*
- OpenAI API Documentation: *OpenAI API Platform Documentation*